

Code Explanation Line by Line (Easy + Detailed)

```
import tensorflow as tf
```

Explanation:

- Imports TensorFlow library.
 - Used for:
 - Deep learning
 - Neural networks
 - Model training
-

```
from tensorflow import keras
```

Explanation:

- Imports Keras from TensorFlow.
 - Keras simplifies neural network coding.
-

```
import matplotlib.pyplot as plt
```

Explanation:

- Imports Matplotlib library.
 - Used to display images and graphs.
-

```
import numpy as np
```

Explanation:

- Imports NumPy library.
 - Used for:
 - Arrays
 - Matrix operations
 - Numerical calculations
-

Load MNIST Dataset

```
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
```

Explanation:

- Loads MNIST handwritten digit dataset.

Dataset contains:

- Training images

- Testing images
- Labels

Variables:

- $X_{train} \rightarrow$ Training images
 - $y_{train} \rightarrow$ Training labels
 - $X_{test} \rightarrow$ Testing images
 - $y_{test} \rightarrow$ Testing labels
-

Normalize Data

$X_{train}, X_{test} = X_{train} / 255.0, X_{test} / 255.0$

Explanation:

- Pixel values originally:

0 to 255

After normalization:

0 to 1

Purpose:

- Faster training
 - Better accuracy
 - Stable learning
-

Build CNN Model

```
model = keras.Sequential([
```

Explanation:

- Creates Sequential CNN model.
 - Layers arranged one after another.
-

First Convolution Layer

```
keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
```

Explanation:

- Adds convolution layer.

Parameters:

- 32 $\rightarrow$ number of filters
- (3,3) $\rightarrow$ filter size

- relu → activation function
- input_shape=(28,28,1) → image size

Purpose:

- Extract image features.
-

ReLU Activation

Formula:

$$ReLU(x) = \max(0, x)$$

If value < 0:

0

Else:

same value

MaxPooling Layer

keras.layers.MaxPooling2D(2,2),

Explanation:

- Reduces image dimensions.
- Keeps important features.

Pooling size:

2 × 2

Second Convolution Layer

keras.layers.Conv2D(64, (3,3), activation='relu'),

Explanation:

- Adds second convolution layer.
- Uses:
 - 64 filters
 - 3×3 kernel

Extracts deeper features.

Second Pooling Layer

keras.layers.MaxPooling2D(2,2),

Explanation:

- Again reduces feature map size.
-

Flatten Layer

`keras.layers.Flatten()`,

Explanation:

- Converts 2D matrix into 1D vector.

Needed before Dense layer.

Dense Hidden Layer

`keras.layers.Dense(128, activation='relu')`,

Explanation:

- Fully connected hidden layer.
- Contains:

128 neurons

Learns complex image patterns.

Output Layer

`keras.layers.Dense(10, activation='softmax')`

Explanation:

- Output layer contains:

10 neurons

Because digits:

0 to 9

Softmax gives probability for each digit.

Compile Model

`model.compile(`

Explanation:

- Configures model before training.
-

`optimizer='adam',`

Explanation:

- Adam optimizer updates weights efficiently.

```
loss='sparse_categorical_crossentropy',
```

Explanation:

- Loss function for multi-class classification.

Measures prediction error.

```
metrics=['accuracy']  
)
```

Explanation:

- Accuracy used to measure performance.
-

Train Model

```
model.fit(
```

Explanation:

- Starts model training.
-

```
X_train,  
y_train,
```

Explanation:

- Training images and labels.
-

```
epochs=5,
```

Explanation:

- Entire dataset trained 5 times.
-

```
validation_data=(X_test, y_test)  
)
```

Explanation:

- Checks model performance during training using test data.
-

Evaluate Model

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
```

Explanation:

- Tests model on unseen data.

- Returns:
 - Loss
 - Accuracy
-

Print Accuracy

```
print("Test Accuracy:", test_accuracy)
```

Explanation:

- Displays final testing accuracy.

Example:

Test Accuracy: 0.98

Meaning:

- 98% correct predictions.
-

Random Sample Selection

```
index = np.random.randint(0, len(X_test))
```

Explanation:

- Selects random image from test dataset.
-

Display Image

```
plt.imshow(X_test[index], cmap='gray')
```

Explanation:

- Displays selected digit image in grayscale.
-

```
plt.title(f"True Label: {y_test[index]}")
```

Explanation:

- Displays actual digit label.
-

```
plt.show()
```

Explanation:

- Shows image window.
-

Predict Digit

```
prediction = np.argmax(
```

Explanation:

- Finds index with highest probability.
-

```
model.predict(X_test[index].reshape(1, 28, 28, 1))
```

Explanation:

- Predicts digit from image.

reshape() converts image into CNN input format.

```
print("Predicted Label:", prediction)
```

Explanation:

- Displays predicted digit.

Example:

Predicted Label: 5

What Happens Internally in This Code? (Detailed Explanation)

This program implements handwritten digit recognition using Convolutional Neural Network (CNN) with TensorFlow and Keras. First, the MNIST dataset is loaded, which contains grayscale images of handwritten digits from 0 to 9. The dataset is divided into training images and testing images. Each image has size 28×28 pixels. The pixel values are normalized between 0 and 1 by dividing them by 255, which improves training efficiency and model performance.

Next, a CNN model is created using Sequential architecture. The model contains convolution layers, max pooling layers, flatten layer, and dense layers. Convolution layers extract important image features such as edges, shapes, and curves. MaxPooling layers reduce image dimensions and computation cost. Flatten layer converts 2D feature maps into a 1D vector, and Dense layers perform final classification of digits.

The model is then compiled using Adam optimizer and sparse categorical crossentropy loss function. Adam optimizer updates neural network weights efficiently, while the loss function calculates prediction error for multi-class classification. During training, forward propagation calculates predictions, and backpropagation updates weights repeatedly to reduce error and improve accuracy. The dataset is trained for 5 epochs.

After training completes, the model is evaluated using unseen testing data. The program then selects a random handwritten digit image from the test dataset, displays it using Matplotlib, and predicts the digit using the trained CNN model. Finally, the predicted digit label and test accuracy are displayed, showing how accurately the model recognizes handwritten digits.